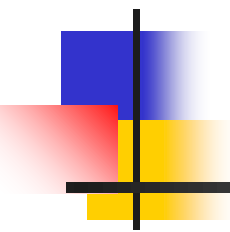
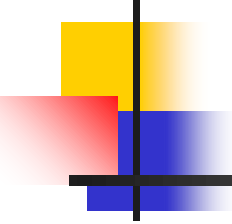


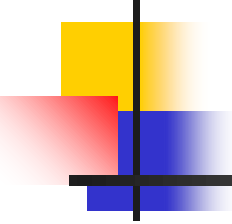
Context-Free Languages & Grammars (CFLs & CFGs)





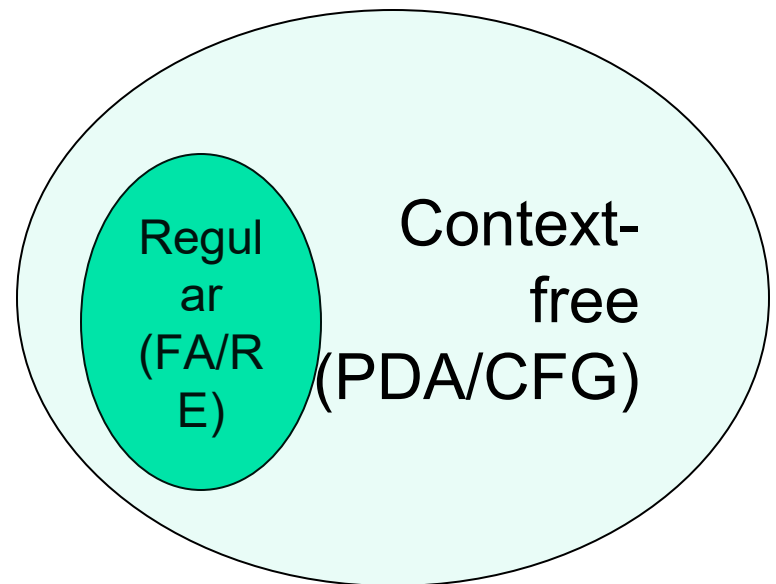
Not all languages are regular

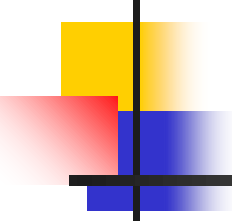
- So what happens to the languages which are not regular?
- Can we still come up with a language recognizer?
 - i.e., something that will accept (or reject) strings that belong (or do not belong) to the language?



Context-Free Languages

- A language class larger than the class of regular languages
- Supports natural, recursive notation called “context-free grammar”
- Applications:
 - Parse trees, compilers
 - XML





An Example

- A palindrome is a word that reads identical from both ends
 - E.g., $\overrightarrow{\text{madam}} \overleftarrow{\text{madam}}$, $\overrightarrow{\text{redivider}} \overleftarrow{\text{redivider}}$, $\overrightarrow{\text{malayalam}} \overleftarrow{\text{malayalam}}$, $\overrightarrow{010010010} \overleftarrow{010010010}$
- Let $L = \{ w \mid w \text{ is a binary palindrome} \}$
- Is L regular?
 - No.
 - Proof:
 - Let $w = 0^N 1 0^N$ (assuming N to be the p/l constant)
 - By Pumping lemma, w can be rewritten as xyz , such that xy^kz is also L (for any $k \geq 0$)
 - But $|xy| \leq N$ and $y \neq \epsilon$
 - $\implies y = 0^+$
 - $\implies xy^kz$ *will NOT* be in L for $k=0$
 - $\implies$ Contradiction

But the language of palindromes...

is a CFL, because it supports recursive substitution (in the form of a CFG)

- This is because we can construct a “grammar” like this:

- Productions
1. $A \Rightarrow \varepsilon$
 2. $A \Rightarrow 0$
 3. $A \Rightarrow 1$
 4. $A \Rightarrow 0A0$
 5. $A \Rightarrow 1A1$

Terminals

Variable or non-terminal

Same as:

$A \Rightarrow 0A0 \mid 1A1 \mid 0 \mid 1 \mid$

ε

How does this grammar work?

How does the CFG for palindromes work?

An input string belongs to the language (i.e., accepted) iff it can be generated by the CFG

- Example: $w=01110$
- G can generate w as follows:

1. $A \Rightarrow 0A0$
2. $\Rightarrow 01A10$
3. $\Rightarrow 01110$

G :

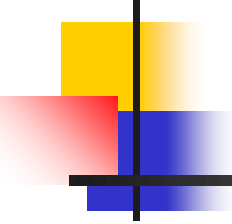
$A \Rightarrow 0A0 \mid 1A1 \mid 0 \mid 1 \mid$

ϵ

Generating a string from a grammar:

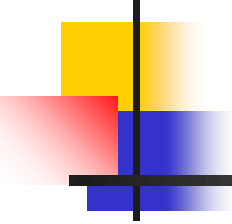
1. Pick and choose a sequence of productions that would allow us to generate the string.

2. At every step, substitute one variable with one of its productions.



Informal Comments About CFG

- A *context-free grammar* is a notation for describing languages.
- It is more powerful than finite automata or RE's, but still cannot define all possible languages.
- Useful for nested structures, e.g., parentheses in programming languages.
- Basic idea is to use “variables” to stand for sets of strings (i.e., languages).
- These variables are defined recursively, in terms of one another.
- Recursive rules (“productions”) involve only concatenation.
- Alternative rules for a variable allow union.



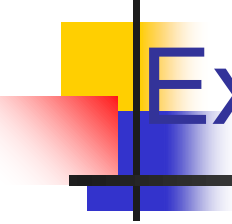
Context-Free Grammar: Definition

- A context-free grammar $G=(V,T,P,S)$, where:
 - V : set of variables or non-terminals
 - T : set of terminals (= alphabet $\cup \{\epsilon\}$)
 - P : set of *productions*, each of which is of the form
 $V \Rightarrow \alpha_1 \mid \alpha_2 \mid \dots$
 - Where each α_i is an arbitrary string of variables and terminals
 - $S \Rightarrow$ start variable

CFG for the language of binary palindromes:

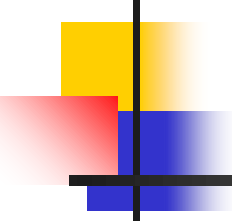
$G=(\{A\},\{0,1\},P,A)$

$P: A \Rightarrow 0 A 0 \mid 1 A 1 \mid 0 \mid 1 \mid \epsilon$



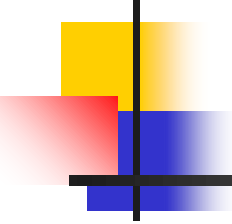
Example #1: CFG for $\{ 0^n 1^n \mid n \geq 1 \}$

- Productions:
 - $S \rightarrow 01$
 - $S \rightarrow 0S1$
- **Basis**: 01 is in the language.
- **Induction**: if w is in the language, then so is $0w1$.



More examples

- Parenthesis matching in code
- Syntax checking
- In scenarios where there is a general need for:
 - Matching a symbol with another symbol, or
 - Matching a count of one symbol with that of another symbol, or
 - Recursively substituting one symbol with a string of other symbols

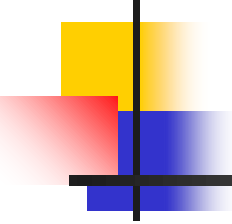


Example #2

- Language of balanced paranthesis
e.g., $()(((((())((()))))((()))))\dots$
- CFG?

$\begin{array}{l} \underline{G}: \\ S \Rightarrow (S) \mid SS \mid \varepsilon \end{array}$

How would you “interpret” the string “ $(((((())((()))))((())))$ ” using this grammar?

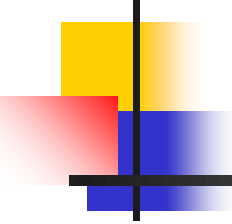


Example #3

- A grammar for $L = \{0^m 1^n \mid m \geq n\}$
- CFG?

G:
 $S \Rightarrow 0S1 \mid A$
 $A \Rightarrow 0A \mid \varepsilon$

How would you interpret the string “00000111”
using this grammar?



Example #4

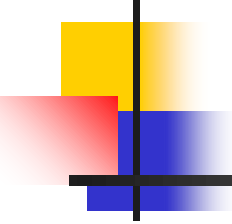
A program containing **if-then(-else)** statements

if *Condition* **then** *Statement* **else** *Statement*

(Or)

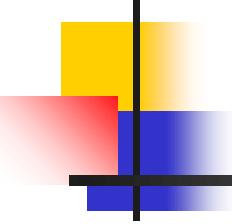
if *Condition* **then** *Statement*

CFG?



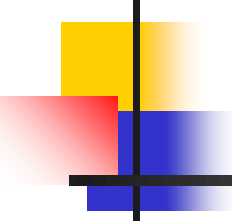
More examples

- $L_1 = \{0^n \mid n \geq 0\}$
- $L_2 = \{0^n \mid n \geq 1\}$
- $L_3 = \{0^i 1^j 2^k \mid i=j \text{ or } j=k, \text{ where } i, j, k \geq 0\}$
- $L_4 = \{0^i 1^j 2^k \mid i=j \text{ or } i=k, \text{ where } i, j, k \geq 1\}$

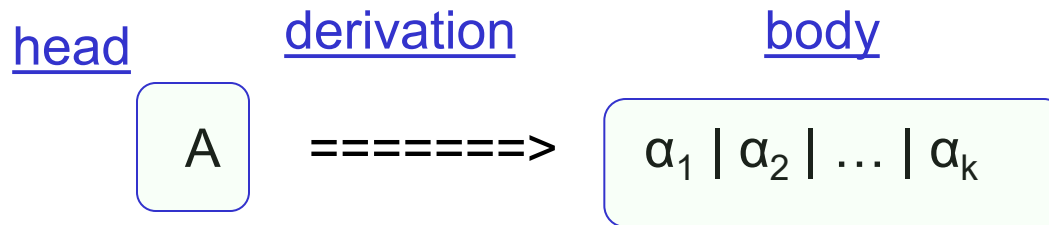


Applications of CFLs & CFGs

- Compilers use parsers for syntactic checking
- Parsers can be expressed as CFGs
 1. Balancing parenthesis:
 - $B \Rightarrow BB \mid (B) \mid \textit{Statement}$
 - $\textit{Statement} \Rightarrow \dots$
 2. If-then-else:
 - $S \Rightarrow SS \mid \textit{if Condition then Statement else Statement} \mid \textit{if Condition then Statement} \mid \textit{Statement}$
 - $\textit{Condition} \Rightarrow \dots$
 - $\textit{Statement} \Rightarrow \dots$
 3. C parenthesis matching $\{ \dots \}$
 4. Pascal *begin-end* matching
 5. YACC (Yet Another Compiler-Compiler)

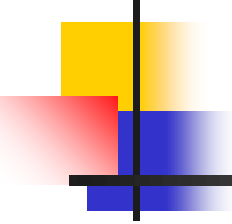


Structure of a production



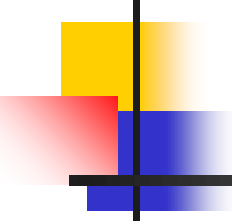
The above is same as:

1. $A \Longrightarrow \alpha_1$
2. $A \Longrightarrow \alpha_2$
3. $A \Longrightarrow \alpha_3$
- ...
- K. $A \Longrightarrow \alpha_k$



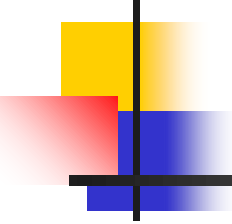
CFG conventions

- A *production* has the form $\text{variable} \rightarrow \text{string of variables and terminals}$.
- **Convention:**
 - $A, B, C, \dots$ are variables.
 - $a, b, c, \dots$ are terminals.
 - $\dots, X, Y, Z$ are either terminals or variables.
 - $\dots, w, x, y, z$ are strings of terminals only.
 - $\alpha, \beta, \gamma, \dots$ are strings of terminals and/or variables.



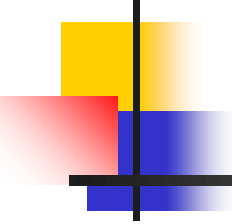
Example: Formal CFG

- Here is a formal CFG for $\{ 0^n 1^n \mid n \geq 1 \}$.
- Terminals = $\{0, 1\}$.
- Variables = $\{S\}$.
- Start symbol = S .
- Productions =
 $S \rightarrow 01$
 $S \rightarrow 0S1$



Derivations – Intuition

- We *derive* strings in the language of a CFG by starting with the start symbol and replacing repeatedly some variable A by the right side of one of its productions.
 - That is, the “productions for A ” are those that have A on the left side of the $\rightarrow$.



Derivations – Formalism

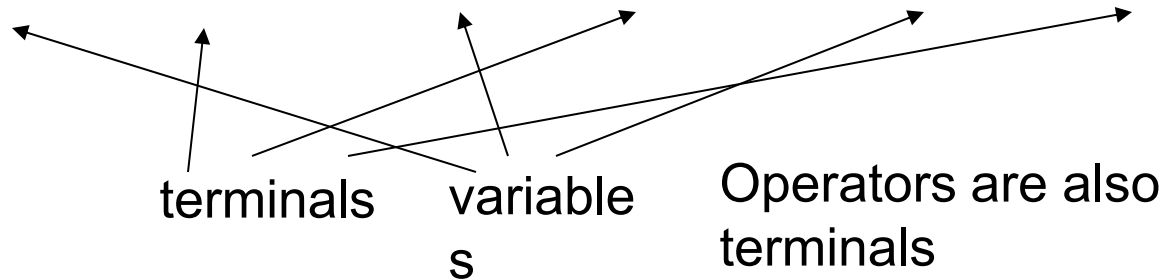
- We say $\alpha A \beta \Rightarrow \alpha \gamma \beta$ if $A \rightarrow \gamma$ is a production.

- **Example:** $S \rightarrow 01$; $S \rightarrow 0S1$.

- $S \Rightarrow 0S1 \Rightarrow 00S11 \Rightarrow 000111$.

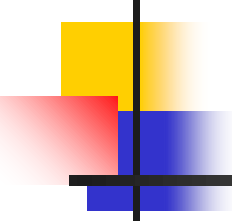
Syntactic Expressions in Programming Languages

*result = a*b + score + 10 * distance + c*



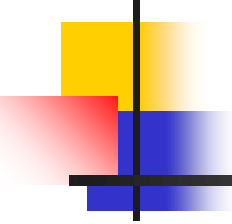
Regular languages have only terminals

- Reg expression = $[a-z][a-z0-1]^*$
- If we allow only letters a & b, and 0 & 1 for constants (for simplification)
 - Regular expression = $(a+b)(a+b0+1)^*$



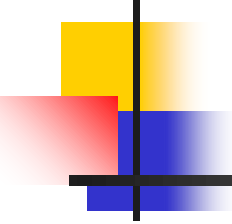
Simple Expressions...

- We can write a CFG for accepting simple expressions
- $G = (V, T, P, S)$
 - $V = \{E, F\}$
 - $T = \{0, 1, a, b, +, *, (,)\}$
 - $S = \{E\}$
 - P :
 - $E \Rightarrow E + E \mid E * E \mid (E) \mid F$
 - $F \Rightarrow aF \mid bF \mid 0F \mid 1F \mid a \mid b \mid 0 \mid 1$



Generalization of derivation

- Derivation is *head* $\Rightarrow$ *body*
- $A \Rightarrow X$ (A derives X in a single step)
- $A \Rightarrow_G^* X$ (A derives X in a multiple steps)
- Transitivity:
IF $A \Rightarrow_G^* B$, and $B \Rightarrow_G^* C$, THEN $A \Rightarrow_G^* C$



Context-Free Language

- The language of a CFG, $G=(V,T,P,S)$, denoted by $L(G)$, is the set of terminal strings that have a derivation from the start variable S .
 - $L(G) = \{ w \text{ in } T^* \mid S \Rightarrow^*_G w \}$

Left-most & Right-most Derivation Styles

G:

$E \Rightarrow E + E \mid E * E \mid (E) \mid F$
 $F \Rightarrow aF \mid bF \mid 0F \mid 1F \mid \varepsilon$

Derive the string $a^*(ab+10)$ from G:

$E \Rightarrow_G^* a^*(ab+10)$

Left-most
derivation:

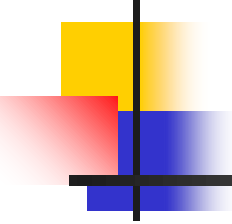
Always
substitute
leftmost
variable

- E
- $\Rightarrow E * E$
- $\Rightarrow F * E$
- $\Rightarrow aF * E$
- $\Rightarrow a * E$
- $\Rightarrow a * (E)$
- $\Rightarrow a * (E + E)$
- $\Rightarrow a * (F + E)$
- $\Rightarrow a * (aF + E)$
- $\Rightarrow a * (abF + E)$
- $\Rightarrow a * (ab + E)$
- $\Rightarrow a * (ab + F)$
- $\Rightarrow a * (ab + 1F)$
- $\Rightarrow a * (ab + 10F)$
- $\Rightarrow a * (ab + 10)$

Right-most
derivation:

Always
substitute
rightmost
variable

- E
- $\Rightarrow E * E$
- $\Rightarrow E * (E)$
- $\Rightarrow E * (E + E)$
- $\Rightarrow E * (E + F)$
- $\Rightarrow E * (E + 1F)$
- $\Rightarrow E * (E + 10F)$
- $\Rightarrow E * (E + 10)$
- $\Rightarrow E * (F + 10)$
- $\Rightarrow E * (aF + 10)$
- $\Rightarrow E * (abF + 0)$
- $\Rightarrow E * (ab + 10)$
- $\Rightarrow F * (ab + 10)$
- $\Rightarrow aF * (ab + 10)$
- $\Rightarrow a * (ab + 10)$



Leftmost vs. Rightmost derivations

Q1) For every leftmost derivation, there is a rightmost derivation, and vice versa. True or False?

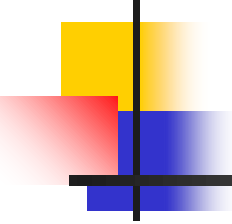
True - will use parse trees to prove this

Q2) Does every word generated by a CFG have a leftmost and a rightmost derivation?

Yes – easy to prove (reverse direction)

Q3) Could there be words which have more than one leftmost (or rightmost) derivation?

Yes – depending on the grammar

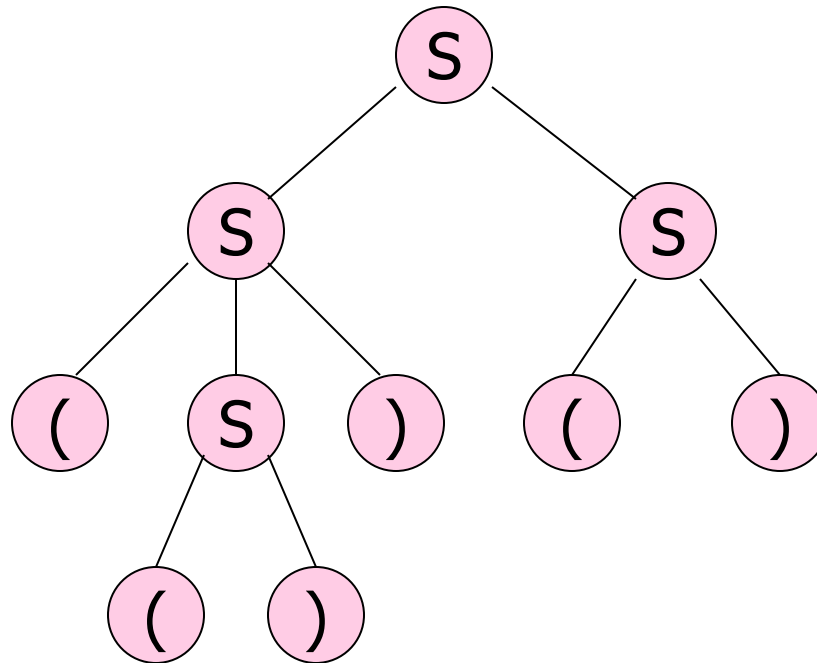


Parse Trees

- *Parse trees* are trees labeled by symbols of a particular CFG.
- **Leaves**: labeled by a terminal or ϵ .
- **Interior nodes**: labeled by a variable.
 - For a production, $A \Rightarrow X_1 X_2 \dots X_k$, then any internal node labeled A has k children which are labeled from $X_1, X_2, \dots, X_k$ from left to right.
- **Root**: must be labeled by the start symbol.

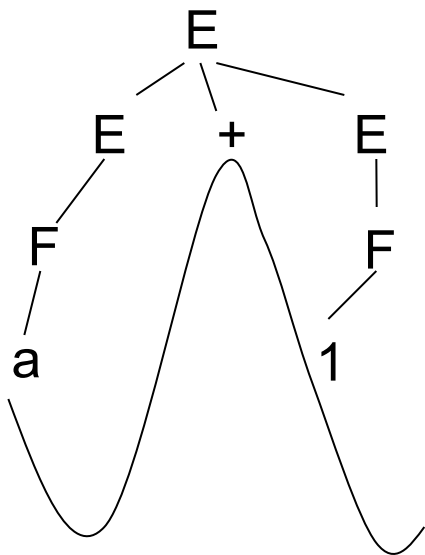
Example: Parse Tree

$S \rightarrow SS \mid (S) \mid ()$



- d of
-
- ```
graph TD; S1((S)) --- NP1((NP)); S1 --- VP((VP)); NP1 --- The1((The)); NP1 --- cat((cat)); VP --- sat((sat)); VP --- PP((PP)); PP --- on((on)); PP --- NP2((NP)); NP2 --- the2((the)); NP2 --- NP3((NP)); NP3 --- mat((mat))
```
- is

# Examples(Yield of a Parse Tree)

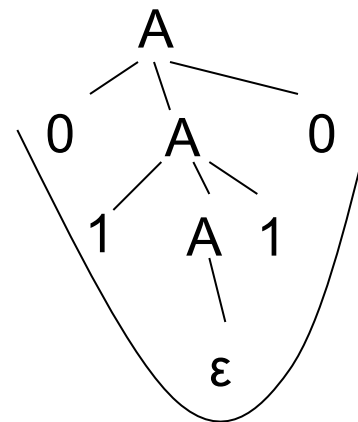


Parse tree for  $a + 1$

G:

$E \Rightarrow E + E \mid E * E \mid (E) \mid F$

$F \Rightarrow aF \mid bF \mid 0F \mid 1F \mid 0 \mid 1 \mid a \mid b$



Parse tree for  $0110$

G:

$A \Rightarrow 0A0 \mid 1A1 \mid 0 \mid 1 \mid$

$\epsilon$

# Ambiguity in CFGs

- A CFG is said to be *ambiguous* if there exists a string which has more than one left-most derivation

Example:

$S \Rightarrow AS \mid \varepsilon$

$A \Rightarrow A1 \mid 0A1 \mid 01$

Input string: 00111

Can be derived in two ways

LM derivation #1:

$S \Rightarrow AS$

$\Rightarrow 0A1S$

$\Rightarrow 0A11S$

$\Rightarrow 00111S$

$\Rightarrow 00111$

LM derivation #2:

$S \Rightarrow AS$

$\Rightarrow A1S$

$\Rightarrow 0A11S$

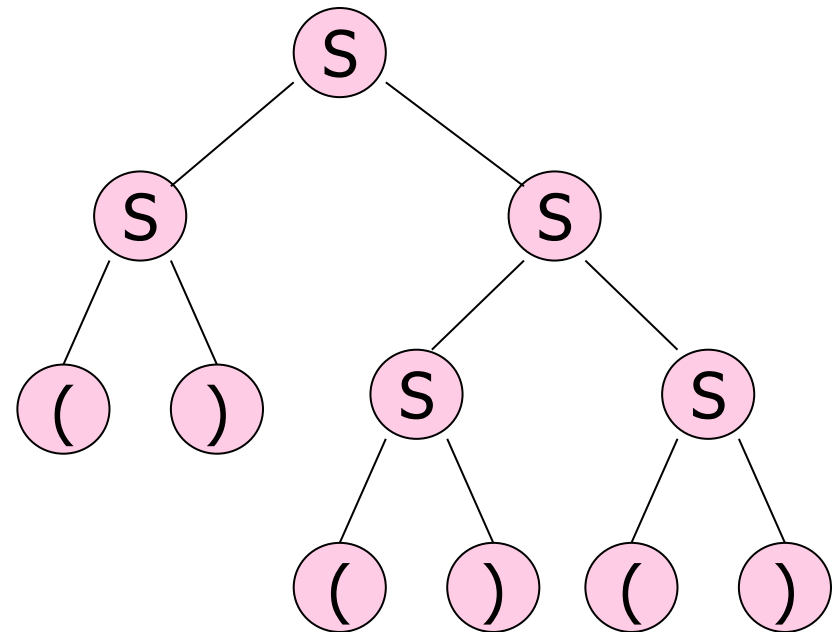
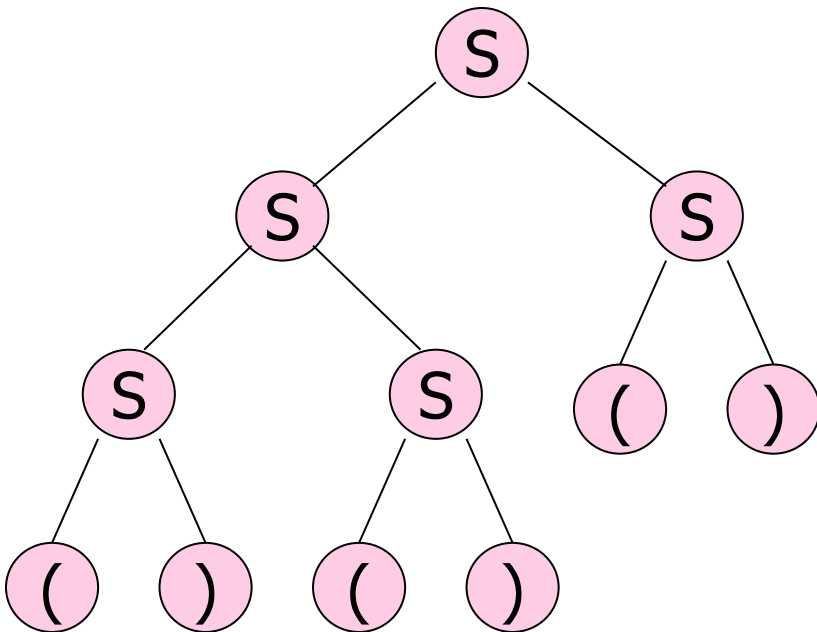
$\Rightarrow 00111S$

$\Rightarrow 00111$

# Example

CFG:  $S \rightarrow SS \mid (S) \mid ()$

Two parse trees for  $()()()$ .



# Why does ambiguity matter?

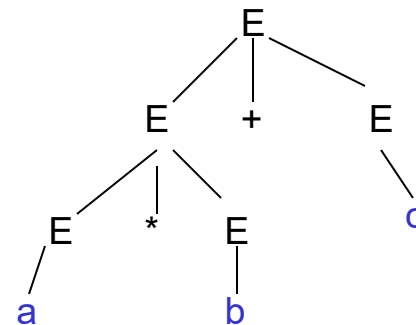
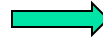
$E \Rightarrow E + E \mid E * E \mid (E) \mid a \mid b \mid c \mid 0 \mid 1$

Values are  
different !!!

*string* =  $a * b + c$

- LM derivation #1:

- $E \Rightarrow E + E \Rightarrow E * E + E$   
 $\Rightarrow a * b + c$

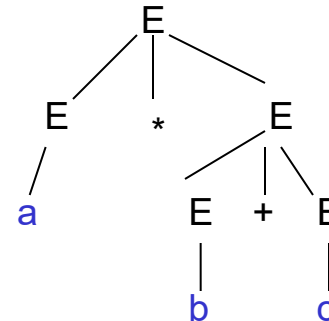


$(a*b)+c$

---

- LM derivation #2

- $E \Rightarrow E * E \Rightarrow a * E \Rightarrow$   
 $a * E + E \Rightarrow a * b + c$



$a*(b+c)$

The calculated value depends on which  
of the two parse trees is actually used.

# Removing Ambiguity in Expression Evaluations

- It MAY be possible to remove ambiguity for some CFLs
  - E.g., in a CFG for expression evaluation by imposing rules & restrictions such as precedence
  - This would imply rewrite of the grammar

- Precedence:  $()$ ,  $*$ ,  $+$

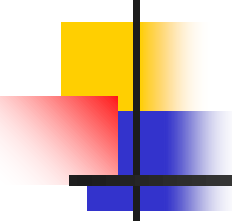
Modified unambiguous version:

$$\begin{aligned} E &\Rightarrow E + T \mid T \\ T &\Rightarrow T * F \mid F \\ F &\Rightarrow I \mid (E) \\ I &\Rightarrow a \mid b \mid c \mid 0 \mid 1 \end{aligned}$$

Ambiguous version:

$$E \Rightarrow E + E \mid E * E \mid (E) \mid a \mid b \mid c \mid 0 \mid 1$$

How will this avoid ambiguity?



# Inherently Ambiguous CFLs

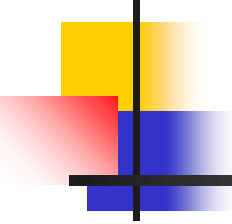
---

- However, for some languages, it may not be possible to remove ambiguity
- A CFL  $L$  is said to be *inherently ambiguous* if every CFG that generates  $L$  is ambiguous.

Example:

- $L = \{ a^n b^n c^m d^m \mid n, m \geq 1 \} \cup \{ a^n b^m c^m d^n \mid n, m \geq 1 \}$
- $L$  is inherently ambiguous
- Why?

Input string:  $a^n b^n c^n d^n$



# One Possible Ambiguous Grammar

---

$S \rightarrow AB \mid CD$

$A \rightarrow 0A1 \mid 01$

$B \rightarrow 2B \mid 2$

$C \rightarrow 0C \mid 0$

$D \rightarrow 1D2 \mid 12$

A generates equal 0's and 1's

B generates any number of 2's

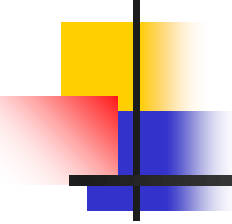
C generates any number of 0's

D generates equal 1's and 2's

And there are two derivations of every string with equal numbers of 0's, 1's, and 2's. E.g.:

$S \Rightarrow AB \Rightarrow 01B \Rightarrow 012$

$S \Rightarrow CD \Rightarrow 0D \Rightarrow 012$



# Summary

---

- Context-free grammars
- Context-free languages
- Productions, derivations, recursive inference, parse trees
- Left-most & right-most derivations
- Ambiguous grammars
- Removing ambiguity